

Requirement Understanding Document (RUD)

Project: MediAssist AI – Enterprise Healthcare Agentic AI Copilot

1. Introduction

- **Purpose:** To design and implement an enterprise-grade Healthcare AI Copilot that integrates structured, unstructured, and multimodal data for hospital operations.
- **Scope:** The system will unify fragmented hospital systems into one intelligent AI platform, enabling conversational search, workflow automation, multimodal reasoning, and secure enterprise integration.

2. Stakeholders

- **Doctors:** Patient summaries, clinical insights
- **Hospital Administrators:** Compliance, operational analytics
- **Front Desk Staff:** Appointment lookup and scheduling
- **Insurance Teams:** Billing and claims verification
- **Clinical Operations Teams:** Workflow and escalation management

3. Problem Statement

Hospitals face challenges such as:

- Fragmented systems and poor interoperability
- Slow information retrieval
- Manual operations and knowledge silos
- Lack of unified AI interface

4. Objectives

- Build a Healthcare AI Copilot capable of:
 - Conversational enterprise search
 - Agentic orchestration with LangGraph
 - Vision-based multimodal understanding
 - Structured + unstructured retrieval via RAG
 - Secure MCP-based PostgreSQL integration
 - Containerized deployment on Azure

5. Functional Requirements

- **User Interaction:** Chat interface, PDF/image uploads, natural language queries
- **Agents:** Planner, Retriever, MCP, Multimodal, Reasoning, Evaluation
- **Data Retrieval:** Vector search (FAISS/ChromaDB), PostgreSQL queries via MCP tools
- **Multimodal Processing:** Prescription image extraction, lab report analysis
- **Evaluation:** LLM-as-a-Judge for hallucination detection, grounding, and confidence scoring
- **Frontend:** Streamlit app with chat, uploads, dashboards, evaluation metrics

6. Non-Functional Requirements

- Modular, API-driven architecture
- Secure credential management
- Containerized deployment with Docker
- Scalable retrieval pipelines
- Observability and monitoring (LangSmith)
- Error handling and guardrails (prompt injection, unsafe recommendations)

7. Data Requirements

- **Unstructured:** SOPs, policies, discharge summaries
- **Multimodal:** Prescriptions, lab reports, medical images
- **Structured:** PostgreSQL tables (Patients, Admissions, Appointments, Billing, Lab Results, Prescriptions, Insurance, Doctors, Departments, Hospital Branches)
- **Scale:** ~1.1 million records across enterprise datasets

8. Constraints & Assumptions

- Students are not expected to build transactional healthcare systems.
- PostgreSQL must be accessed only via MCP server tools.
- Provided datasets simulate enterprise-scale hospital operations.

9. Acceptance Criteria

- Working demo showcasing:
 - RAG pipeline

- MCP integration
- Multimodal uploads
- Multi-agent orchestration
- Deployment on Azure with Dockerized setup
- Evaluation report including retrieval quality, hallucination analysis, latency, and agent performance

10. Risks & Mitigation

- **Risk:** Hallucinations in AI responses → **Mitigation:** LLM-as-a-Judge evaluation
- **Risk:** Data privacy concerns → **Mitigation:** Guardrails for sensitive information
- **Risk:** System scalability → **Mitigation:** Containerized deployment and modular design

11. Deliverables

- GitHub repository (code, Docker setup, docs)
- Architecture documentation (system flow, agent workflows, MCP integration)
- Working demo (RAG, multimodal, MCP orchestration)
- Azure deployment
- Evaluation report